

# CLS Requests — Technical Guide

How the CLS feature is built • Beacon Performance & Budgeting (R Shiny + Postgres)

## Orientation

Beacon is a single-file R Shiny application. **app.R** holds the UI, every page-building function and the server; **R/database.R** holds all SQL, the startup migration and the data loader; **R/auth.R** holds authentication. Client behaviour lives in **www/app.js** and styling in **www/styles.css**. Both R files are pulled in with `source( . . . , local = TRUE )` at the top of `app.R`, so anything defined in them is visible app-wide.

The app follows a load-everything-then-render model: `load_app_data( )` reads the database into a named list of data frames, pages read from that list, and any write is followed by `refresh_app_data( )`. The CLS feature plugs into that cycle rather than inventing its own persistence path.

## Files touched

| File                     | What the CLS work added                                                                                                                                     |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R/database.R             | Four tables in the startup migration, four loader queries, the write helpers, and the request-type / object / status vocabularies.                          |
| app.R                    | Role helpers, three page functions, the export builders, routing, nav entries and all <code>input\$cls_*</code> observers.                                  |
| www/app.js               | The request-page controller (validation, toggles, autosave trigger), table sorting, collapsible review rows and the <code>data-cls-*</code> click handlers. |
| www/styles.css           | Everything under the <code>.cls-</code> prefix plus two nav-visibility body classes.                                                                        |
| scripts/build_cls_pdf.py | The PDF export renderer (reportlab).                                                                                                                        |
| Dockerfile               | <code>writexl</code> for the Excel export; copies the CLS PDF script.                                                                                       |

## Data model

Four tables in the Postgres budget schema. The first three are the BBMR target schema's CLS\_REQUEST / CLS\_REQUEST\_LINE / CLS\_REQUEST\_POSITION translated to Postgres types; `cls_review` is additive and holds BBMR's decision so the agency's submission is never overwritten. The canonical DDL is `database/schema/target_schema.sql`; `ensure_review_schema( )` migrates existing databases to match.

**Both must be changed together** — `CREATE TABLE IF NOT EXISTS` never alters an existing table, and CI builds from the canonical schema alone. Three CI failures on this feature were exactly this mistake.

| Table                       | Key columns                                                                                                                                                                            | Notes                                                                                                                                                                                                  |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| budget.cls_request          | cls_id PK, plan_service_id FK, request_name, request_type, request_amount, one_time, overall_summary, <b>status</b> , amount_next_fy, amount_2next_fy, <b>created_by</b> , modified_by | One row per request. <b>status</b> replaced the target schema's completed BIT (six states, not two); <b>justified</b> was dropped. <code>created_by</code> / <code>modified_by</code> are user_id FKs. |
| budget.cls_request_line     | line_id PK, cls_id FK, object_category, <b>spend_category</b> , amount, justification, sort_order                                                                                      | One row per expenditure object.                                                                                                                                                                        |
| budget.cls_request_position | pos_id PK, cls_id FK, classification, position_count, estimated_salary, justification, explanation                                                                                     | One row per position request.                                                                                                                                                                          |

| Table             | Key columns                                                                                                                                      | Notes                                                                     |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| budget.cls_review | review_id PK, cls_id FK UNIQUE, analyst_notes, analyst_approval, bbmr_approval, <b>approved_amount</b> , <b>approved_positions</b> , reviewed_by | One review per request; additive by design. evaluation_score was dropped. |

### Migration strategy

Everything is created inside `ensure_review_schema()`, which `app.R` calls once per session start. Statements are written to be safely re-runnable — `CREATE TABLE IF NOT EXISTS`, `ADD COLUMN IF NOT EXISTS`, and a guarded backfill — so an existing database picks up the feature with no manual step. Two hazards were handled explicitly:

- **Columns that already existed with the wrong type.** `modified_by` existed as integer on a development database, which broke a text `COALESCE`. The migration coerces it: `ALTER COLUMN modified_by TYPE text USING modified_by::text`.
- **Cascades that only exist on fresh tables.** `IF NOT EXISTS` never adds a missing `ON DELETE CASCADE`, so on older databases deleting a request failed on a foreign key. `delete_cls_request()` therefore deletes lines, positions and the review row itself inside a transaction rather than relying on the constraint.

### Status is the state machine

`status` drives the workflow and the UI; the target schema's boolean `completed` is kept in sync as a derived value so it stays meaningful for anything downstream that expects it.

```
cls_status_choices <- c("In Progress", "Agency Review", "BBMR Review",
                        "Approved", "Partially Approved", "Denied")

cls_status_is_complete <- function(status)
  status %in% c("BBMR Review", "Approved", "Partially Approved", "Denied")
```

`set_cls_status()` moves one request and writes both columns together. `set_plan_cls_status()` can move a whole plan and takes an `only_from` guard so a sweep never overwrites an already-decided request — it is retained for bulk operations even though the UI now submits one request at a time. A BBMR decision is applied inside `save_cls_review()`: Approved / Partial / Denied map to Approved / Partially Approved / Denied.

## How the pages are built

Pages are plain functions returning htmltools tags. `page_ui()` is a `switch()` that maps a page key to one of them, and it is also where server-side access gating happens — an unauthorised page key is rewritten before anything renders. `output$page` calls `page_ui()` with the current page, the loaded data, the user's roles and any selection state.

```
page_ui <- function(page, db, agency_id, ..., selected_cls_id, cls_review_filters) {
  if (page %in% c("cls_requests", "cls_request_detail") &&
      !can_view_cls_requests(app_roles)) page <- "landing"
  if (identical(page, "cls_review") && !can_review_cls(app_roles)) page <- "landing"
  switch(page,
    cls_requests      = page_cls_requests(db, agency_id, app_roles),
    cls_request_detail = page_cls_request_detail(db, selected_cls_id, app_roles, agency_id),
    cls_review        = page_cls_review(db, app_roles, ...)
  )
}
```

### Roles

Four predicates over the user's app roles decide everything: `can_view_cls_requests()`, `can_edit_cls_requests()`, `can_approve_cls()` (approvers and admins), and `can_review_cls()` (BBMR and admins). They are checked in three places: in `page_ui()`, again inside every mutating observer, and in the nav-visibility message so hidden pages are not merely invisible but unreachable.

### *page\_cls\_requests()* — the list

- Scopes to the agency's current plan: `cls_requests_for_plan()` filters loaded requests by `plan_id`.
- Column headers are buttons carrying `data-cls-sort`; each row carries `data-sort-name/-service/-amount/-status` so sorting happens client-side with no server round-trip.
- The row's hand-off action is chosen by role and status: approvers get *Send to BBMR*, writers get *Submit*, and only when the status makes it valid.
- Amounts render through `cls_format_km()` ( $\$X.XK / \$X.XM$ ); status renders as a chip toned by `cls_status_tone()`.

### *page\_cls\_request\_detail()* — the request

One function serves three modes, decided by whether an id was passed and what the user may do: **new** (empty form plus a Create button), **edit** (prefilled, autosaving, no Save button), and **read-only** (values rendered as text, no add-forms). It is built as three stacked sections:

- **Summary** — name, service, request type, duration toggle, the three fiscal-year amounts, and the word-limited narrative. Labels that need an explanation are built by a small local helper, `cls_labelled_info()`, which emits the label, an info button carrying `data-cls-info`, and a hidden panel rendered after the input so it opens below the field.
- **Request details** — the balance banner, then the add-form, then the object table. The banner text is computed server-side from `request_amount - (line_total + position_total)` and recomputed live in JavaScript as the user types.
- **Position requests** — rendered but hidden behind a checkbox, off unless positions already exist.

Rows carry `data-cls-line-amount` and `data-cls-position-amount` specifically so the client can total them without re-querying.

### *page\_cls\_review()* — the BBMR workspace

- `cls_review_rows()` flattens every request across every agency into one frame: agency label, line/position rollups, and any review fields joined on.
- `cls_review_chart()` emits an inline SVG bar chart — no plotting library. Tooltips are native `<title>` elements, so they work without JavaScript and are readable by assistive tech.
- Requests render as a compact table whose rows are buttons carrying data-cls-rv-toggle; the detail panel below each row is collapsed by default.
- Filters are deliberately *not* live. Applied values are held in `cls_applied_status` / `cls_applied_agency` reactives that only update when *Apply filters* is pressed; *Reset* clears them and pushes the full choice lists back into the inputs.

## Server wiring

Every CLS action is an `observeEvent` on an `input$cls_*` value, and they all follow the same shape: check permission, coerce the id, call the helper inside `tryCatch`, surface any error with `showNotification`, then refresh and notify.

```
observeEvent(input$cls_delete, {
  if (!cls_guard_edit()) return()
  cls_id <- suppressWarnings(as.integer(input$cls_delete$clsId))
  if (is.na(cls_id)) return()
  result <- tryCatch(delete_cls_request(database, cls_id), error = function(e) e)
  if (inherits(result, "error")) { showNotification(conditionMessage(result), type = "error"); r
  eturn() }
  refresh_app_data()
  showNotification("CLS request deleted.", type = "message")
}, ignoreInit = TRUE)
```

## Selection state

`current_cls_id()` is the single source of truth for which request the detail page is showing; NA means “new”. Opening a row sets it and pushes a page change; creating a request sets it to the new id so the user lands on the saved request. `cls_pending_submit()` holds the id and mode behind a confirmation dialog so the confirm handler acts on exactly the request that was clicked.

## Autosave

Typing in the summary section schedules a debounced (~0.9s) `cls_autosave` event. The observer writes through `update_cls_request()` stamped with the current user and records the time in `cls_last_save()`, which feeds the indicator. It deliberately does **not** call `refresh_app_data()` — that would re-render the form under the user's cursor. The list picks up the changes instead when they navigate away, via a check in the `input$current_page` observer.

## Client behaviour (www/app.js)

One delegated `click` listener handles every `data-cls-*` attribute and forwards to Shiny with `setInputValue(..., {priority: "event"})`, each payload carrying a nonce so repeated identical actions still fire. Destructive actions confirm first.

| Function                                    | Responsibility                                                                                            |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>clsValidate()</code>                  | Umbrella: word count, balance banner, red required fields.                                                |
| <code>clsUpdateWordCount()</code>           | 150-word limit; reports the exact overage in red.                                                         |
| <code>clsUpdateRemaining()</code>           | Recomputes FY28 minus (objects + positions); swaps the banner between neutral, green and red.             |
| <code>clsApplyOneTime() / ...Label()</code> | Hides and clears FY29/FY30; flips the label between One-Time and Recurring.                               |
| <code>clsApplyPositionsToggle()</code>      | Shows or hides the positions body; clears the entry fields when switched off.                             |
| <code>clsSyncTitle()</code>                 | Mirrors the request name into the page heading and the intro sentence as it is typed.                     |
| <code>clsScheduleAutosave()</code>          | Debounces the autosave event and shows ‘Saving...’.                                                       |
| <code>clsRequestIsComplete()</code>         | Whether the request balances; gates the warning shown on the way out. A complete request leaves silently. |

| Function                                                      | Responsibility                                                                                                                                |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>clsSummaryGaps()</code>                                 | Which mandatory summary fields are empty, by label. Blocks navigation. Skips hidden fields, so a one-time request is not asked for FY29/FY30. |
| <code>clsNumberValue()</code> / <code>clsGroupDigits()</code> | Read and format separated numbers. Anything reading an amount field must use these — <code>parseFloat("1,000")</code> is 1.                   |
| <code>clsApplyCheckDropdown()</code>                          | Pushes a filter dropdown's held ticks to Shiny as one change event.                                                                           |
| <code>clearStateOverlays()</code>                             | Removes an orphaned modal backdrop on navigation.                                                                                             |

### *The money input binding*

Amount and count fields are **not** `numericInput()`. A native `<input type="number">` cannot display a thousands separator — the value is invalid the moment a comma appears. `cls_money_input()` emits the markup Shiny's own `numericInput` would, with a text input classed `cls-money-input`, and `app.js` registers a custom input binding (`beacon.clsMoneyInput`) that strips separators and truncates decimals on the way to the server, re-groups them on the way back, blocks '.' on keydown, and debounces at 350ms. Round-trip: "1,234,567" → 1234567; "1234.99" → 1234; "" and "abc" → null.

### *Batched filter dropdowns*

Every tick in a CLS Review filter used to reach Shiny on its own, re-rendering the table and closing the panel mid-selection. A **capture-phase** change listener on document now stops the event before it reaches the container Shiny's checkbox-group binding listens on, marks the panel dirty and updates the summary locally. Applying dispatches one synthetic change — the binding reads the whole container, not the box that fired — guarded by a flag so the capture listener lets that one through.

Re-initialisation matters: Shiny replaces the page's DOM on every render, so `clsInitPage()` is re-run on the `shiny:value` event to reapply toggles and validation to the new nodes.

## Exports

- **Excel.** `cls_export_sheets()` returns a named list of three data frames — Request summary, Line items, Personnel — and `writexl::write_xlsx()` turns each into a worksheet. Amounts stay numeric so Excel can total them.
- **PDF.** `cls_export_payload()` builds nested JSON (request → lines, positions), writes it to a temp file, and `scripts/build_cls_pdf.py` renders it with `reportlab` through the existing `PLAN_EXPORT_PYTHON` virtualenv. The R side treats a missing or empty output file as an error rather than serving a broken download.

## Conventions worth keeping

- Permission is checked at render **and** at mutation — never rely on a hidden control.
- SQL is always parameterised; no string interpolation of user input.
- Vocabularies (types, objects, statuses) live in `R/database.R` as single sources of truth and are read by the UI, so a label changes in exactly one place.
- Client-side work is limited to presentation and validation; the database remains the authority.
- Migrations are re-runnable and defensive, because they execute against databases that already exist.